

LLM Chat History — Scenario 22: Sliding Puzzle

User:

I have a sliding puzzle project with `main.py`, `game.py`, and `puzzle.py`. The README requires fixing solvability, completing the puzzle lifecycle, adding 3×3/4×4/5×5 modes, and fixing valid/invalid move feedback. I have forked the repository and need step-by-step guidance.

LLM:

First, clone the forked repository, open it in VS Code, and run the original program with:

```
python main.py
```

Before changing anything, inspect several fresh puzzle boards and reproduce the original behaviour.

User:

I inspected several boards and shared `puzzle.py`.

LLM:

The original defect is in:

```
random.shuffle(tiles)
```

This creates arbitrary permutations, including unreachable sliding-puzzle states.

For Task 1, generate the solved board first and scramble it using legal blank moves. This guarantees that every generated board is reachable from the solved state.

Implemented:

- Removed arbitrary shuffling.
 - Started from the solved arrangement.
 - Applied random legal blank moves to scramble the board.
-

User:

I recorded the 10-second video before making changes.

LLM:

Proceed with Task 1, test several fresh boards, and commit the change separately.

User:

Task 1 works.

LLM:

For Task 2, inspect `game.py` and ensure the game detects the solved arrangement and terminates cleanly. Add a finished-state flag so no further commands can modify the puzzle after completion.

Implemented:

- Added finished state.

- Detects puzzle.solved().
 - Prints Solved!.
 - Ends the game after completion.
-

User:

Task 2 works. Move to Task 3.

LLM:

For Task 3, add size selection for:

3×3

4×4

5×5

The selected size is passed to Puzzle(size). Move count and timer are initialized consistently for the selected puzzle.

Implemented:

- Added size selection.
 - Added 3×3, 4×4 and 5×5 modes.
 - Preserved move tracking.
 - Preserved elapsed-time tracking.
-

User:

Task 3 works. Move to Task 4.

LLM:

For Task 4, ensure that only an actual successful slide increments the move count and produces success feedback. Invalid moves must not increment the counter.

Implemented:

```
if self.puzzle.move(key):
```

```
    self.moves += 1
```

```
    print("Move successful.")
```

```
else:
```

```
    print("That move is not possible.")
```

This means valid moves increment the counter, while invalid moves do not.

User:

I tested the final version and recorded the 10-second after video.

LLM:

Final testing should cover:

- 3×3, 4×4 and 5×5 boards
- Valid moves
- Invalid moves
- Move-count behaviour
- Timer behaviour
- Invalid commands
- Solving a small board
- Quit command

The final submission should contain the before/after videos, updated code, and LLM chat history as required by the lab instructions.